

LAB # 09

Construct classes and objects in java using Netbeans IDE. (methods)

Objective

Get familiar with concept of classes and objects in java.

Lab Goals

- Gain a fundamental understanding of classes, methods and attributes in Java.
- Learn how to create simple classes and declare objects.
- Explore the use of the `new` keyword to instantiate objects.
- Define methods within a class to perform specific tasks.
- Understand the usage of the `this` keyword for referencing current object instances.

Theory

Everything in Java is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an object. The car has attributes, such as weight and color, and methods, such as drive and brake. A Class is like an object constructor, or a "blueprint" for creating objects.

- **Object** – Objects have states and behaviors. Example: A dog has states - color, name, breed as well as behaviors – wagging the tail, barking, eating. An object is an instance of a class.
- **Class** – A class can be defined as a template/blueprint that describes the behavior/state that the object of its type support.

In general, class declarations can include these components, in order:

- **Modifiers:** A class can be public or has default access (Refer this for details).
- **Class name:** The name should begin with a initial letter (capitalized by convention)
- **Body:** The class body surrounded by braces, { }.

Creating a Class in java

To create a class, use the keyword **class**:

```
Public class MyClass{  
  
    //class body  
  
}
```

Creating an Object

In Java, an object is created from a class. We have already created the class named `MyClass`, so now we can use this to create objects.

To create an object of `MyClass`, specify the class name, followed by the object name, and use the keyword **new**:

```
MyClass myObj = new MyClass();
```

Multiple Object

You can create multiple objects of one class:

```
MyClass myObj = new MyClass();  
MyClass myobj1=new MyClass();  
  
MyClass myobj2=new MyClass();
```

A class can contain any of the following variable types.

- **Local variables** – Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables** – Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.
- **Class variables** – Class variables are variables declared within a class, outside any method, with the static keyword.

A class can have any number of methods to access the value of various kinds of methods.

```
public class Puppy {  
    System.out.println("Passed Name is :" + name );  
}  
public static void main(String []args) {  
    // Following statement would create an object myPuppy  
    Puppy myPuppy = new Puppy( "tommy" );  
}  
}
```

Output

```
Passed Name is :tommy
```

```
//Java Program to illustrate how to define a class and fields  
//Defining a Student class.  
class Student{  
    //defining fields  
    int id;//field or data member or instance variable  
    String name;  
    //creating main method inside the Student class  
    public static void main(String args[]){  
        //Creating an object or instance  
        Student s1=new Student();//creating an object of Student  
        //Printing values of the object  
        System.out.println(s1.id);  
        //accessing member through reference variable  
        System.out.println(s1.name);  
    }  
}
```

Output

```
0  
null
```

Defining Methods

Methods are functions defined within a class to perform specific tasks or operations. They can manipulate the state of an object and provide behavior to the class.

Example

```
public void drive() {  
    System.out.println("The car is driving.");  
}
```

Example

```
public class Student {  
  
    String name;  
    int rollno;  
    int age;  
  
    void info(){  
        System.out.println("Name: "+name);  
        System.out.println("Roll Number: "+rollno);  
        System.out.println("Age: "+age);  
    }  
  
    public static void main(String[] args) {  
        Student student = new Student();  
  
        // Accessing and property value  
        student.name = "Babar";  
        student.rollno = 101;  
        student.age = 20;  
  
        // Calling method  
        student.info();  
    }  
}
```

Output

```
Name: Babar  
Roll Number: 101  
Age: 20
```

Exercise

- A. Write a program with a method named `getTotal` that accepts two integers as an argument and return its sum. Call this method from `main( )` and print the results.
- B. Write a program that create two classes one is `Power_law` another is the main method class. `Power_law` class contains `current`, `voltage` and `power` fields in integer datatype, and a method to calculate power and display the output.
- C. Assign and print the roll number, phone number and address of two students having names "Babar" and "Danish" respectively by creating two objects of class 'Student'.
- D. Add two distances in inch-feet by creating a class named 'AddDistance'